

AIVOA.AI

Product Description, Tech Stack & End-to-End Workflow

PharmaOne-AI / AIVOA.AI Connected QMS

Document 1 of 5

Professional presentation edition • Content preserved from the supplied source documents

AIVOA.AI — PRODUCT DESCRIPTION & QMS UNDERSTANDING

Product: PharmaOne-AI / AIVOA.AI Connected QMS

AIVOA.AI is positioned around an AI-assisted Quality Management System workflow for life-sciences manufacturing. The Round 2 scope upgrades the existing deviation-focused product into a connected QMS workflow using an API manufacturing example. The implementation is explicitly not intended to become a full enterprise QMS.

Purpose of the Product

The product is designed to make the quality-event lifecycle connected and traceable. The central workflow is Deviation → Investigation → Root Cause → CAPA → Effectiveness → Closure, with manufacturing and downstream quality context connected to the same records.

API Manufacturing Context

The supplied challenge uses a realistic API manufacturing chain: Raw Material → Manufacturing → In-Process Checks → Deviation → Investigation → CAPA → Batch Release → Complaint / Recall → Supplier Quality. The project uses this chain to demonstrate how quality processes connect.

Core QMS Workflow

Stage	Purpose / connection
Raw Material	Approved material and supplier context.
Manufacturing Batch	Manufacturing record and process context.
In-Process Check	Monitor defined process parameters and identify unexpected or out-of-limit events.
Deviation	Formally record and control the quality event.
Investigation	Collect evidence, tasks, records and determine what happened.
Root Cause / 5 Whys	Analyze contributing factors and confirm the root cause.
CAPA	Define corrective and preventive actions.
Effectiveness	Verify whether the CAPA worked.
Closure	Authorized closure after required quality gates are satisfied.
Batch Release	Connect the quality history to QA release/disposition context.

Example Quality Event

The demonstration uses Paracetamol API batch API-2026-041. An in-process reaction temperature is 84°C while the approved range is 76–80°C. This out-of-limit process event creates deviation DEV-2026-018.

AI Role

AI assists with extraction, evidence retrieval, summaries, investigation suggestions, root-cause drafts and CAPA recommendations. AI output is treated as a recommendation, not as the authoritative quality decision.

Human Oversight

Human users review or accept/edit/reject AI recommendations. Severity, impact, root cause, CAPA actions, effectiveness and closure remain reviewable human decisions.

Traceability

Important records are linked so users can move between the batch, in-process check, deviation, investigation, root cause, CAPA, effectiveness and batch-release context. AI recommendations are intended to retain evidence, model/prompt metadata, reviewer decision and timestamp.

Connected Context

Complaint/Recall and Supplier Quality are connected to the main lifecycle as supporting context. The Round 2 design describes these as lightweight context rather than separate deep workflow implementations.

Technology Direction

The supplied Round 2 documents preserve the existing FastAPI + Supabase/PostgreSQL + pgvector + LangGraph/Groq direction, with React/Redux on the frontend and Pydantic validation for structured outputs.

Source Basis

Prepared from the supplied AIVOA.AI Product Understanding Challenge and the five AIVOA Round 2 design documents. This PDF describes the documented product scope and does not add unsupported capabilities.

FastAPI authenticates the request. 7. Tenant/site permissions are resolved. 8. Router validates request schema. 9. Service executes business logic. 10. Repository reads/writes PostgreSQL. 11. If AI is needed, service invokes LangGraph. 12. AI graph parses/retrieves/reasons/validates. 13. AI result returns to service. 14. Service persists approved/final data. 15. Audit event is written. 16. API returns a typed response. 17. Redux updates state. 18. UI re-renders the relevant workflow step.

This separation keeps business rules out of React components and keeps AI orchestration out of database models. Humanity has suffered enough from 900-line frontend components that secretly contain half the backend.

5. Authentication and Tenant Request Flow

Browser ↓ Supabase Auth session ↓ Access token ↓ Authorization: Bearer <token> ↓ FastAPI dependency ↓ Validate JWT / user identity ↓ Load company membership ↓ Resolve site permissions ↓ Attach: current_user company_id site_id roles ↓ Service/repository queries automatically scoped by company_id

6. API Conventions

Base: /api/v1 Examples: GET /batches GET /batches/{batch_id} POST /batches GET /deviations POST /deviations GET /deviations/{deviation_id} PATCH /deviations/{deviation_id} POST /deviations/{id}/start-investigation POST /investigations/{id}/root-cause POST /investigations/{id}/capa POST /capas/{id}/effectiveness POST /deviations/{id}/close POST /ai/deviation/process POST /ai/recommendations/{id}/review

6.1 API behavior rules

Use UUIDs internally.

Use readable reference numbers such as DEV-2026-018 in the UI.

Return ISO timestamps.

Return consistent error objects.

Use pagination for lists.

Use optimistic concurrency/version checks where records can be edited by multiple users.

Require idempotency for expensive AI processing endpoints where repeated submissions are possible.

Never trust company_id supplied by the browser. Derive tenant context from authenticated membership.

7. Error Contract

```
{ "error": { "code": "DEVIATION_NOT_FOUND", "message": "Deviation was not found.", "request_id": "req-uuid", "details": {} } }
```

8. Database Interaction Flow

FastAPI service ↓ SQLAlchemy repository ↓ PostgreSQL / Supabase ↓ Transaction ■■■■ business record ■■■■ linked record updates ■■■■ audit event ↓ COMMIT ↓ API response

Use transactions around state transitions. For example, closing a deviation should update the final status and write the corresponding audit event as one logical operation.

9. Document Upload Workflow

1. User selects PDF/image. 2. Browser validates file type/size. 3. Multipart request → FastAPI. 4. Auth + tenant check. 5. File stored in Supabase Storage. 6. Storage reference persisted. 7. Background/AI workflow receives file reference. 8. Parser attempts text extraction. 9. If scanned, OCR fallback. 10. Text normalized. 11. Structured extraction. 12. Pydantic validation. 13. RAG retrieval. 14. Recommendation generation. 15. Review package returned. 16. User reviews. 17. Final values persisted.

9.1 File handling rules

Never trust filename or MIME type alone.

Enforce maximum upload size.

Store files under tenant-scoped paths.

Use opaque storage identifiers rather than exposing internal paths.

Record document hash for traceability.

Do not send entire unrelated files to the LLM.

Keep source document reference attached to extracted evidence.

10. CRUD Workflow Example: Create a Deviation Manually

UI ↓ POST /api/v1/deviations ↓ Auth + tenant check ↓ Pydantic request validation ↓ DeviationService.create() ↓ Repository INSERT ↓ AuditService.record("deviation.created") ↓ COMMIT ↓ Return DeviationResponse ↓ Redux store update ↓ Navigate to /deviations/{id}

11. AI Request Workflow

UI ↓ POST /api/v1/ai/deviation/process ↓ Auth / tenant ↓ AI service ↓ LangGraph ■■■ parse ■■■ OCR if needed ■■■ extract ■■■ validate ■■■ retrieve ■■■ rules ■■■ LLM ■■■ validate ↓ AI recommendation object ↓ AI interaction persistence ↓ API response ↓ Review UI ↓ Accept/Edit/Reject ↓ Final business API ↓ Deviation record + audit event

12. Full QMS Lifecycle Request Sequence

BATCH ■■■ GET /batches/{id} ■ ▼ IN-PROCESS CHECK ■■■ POST /in-process-checks ■ ▼ DEVIATION ■■■ POST /ai/deviation/process ■■■ POST /ai/recommendations/{id}/review ■■■ POST /deviations ■ ▼ INVESTIGATION ■■■ POST /deviations/{id}/start-investigation ■■■ POST /investigations/{id}/ai-assist ■■■ PATCH /investigations/{id} ■ ▼ ROOT CAUSE ■■■ POST /investigations/{id}/root-cause/ai-draft ■■■ POST /investigations/{id}/root-cause ■ ▼ CAPA ■■■ POST /investigations/{id}/capa ■■■ POST /capas/{id}/ai-suggest ■■■ PATCH /capa-actions/{id} ■ ▼ EFFECTIVENESS ■■■ POST /capas/{id}/effectiveness ■■■ POST /capas/{id}/effectiveness/ai-summary ■ ▼ CLOSURE ■■■ POST /deviations/{id}/closure-summary ■■■ POST /deviations/{id}/close ■ ▼ BATCH RELEASE ■■■ GET /batch-release/{batch_id}

13. Complete Demo Scenario

Use one connected API manufacturing story throughout the video so the evaluator can see the relationships rather than watching a collection of unrelated CRUD screens.

14. Frontend Route Map

/ /dashboard /manufacturing/batches /manufacturing/batches/:batchId /manufacturing/in-process-checks /quality/deviations /quality/deviations/:deviationId /investigations /investigations/:investigationId /investigations/:investigationId/root-cause /capa /capa/:capaId /capa/:capaId/effectiveness /batch-release/:batchId /complaints /complaints/:complaintId /suppliers /suppliers/:supplierId /audit

15. Redux State Design

store/ ■■■ authSlice ■■■ dashboardSlice ■■■ batchSlice ■■■ ipcSlice ■■■ deviationSlice ■■■ investigationSlice ■■■ rootCauseSlice ■■■ capaSlice ■■■ effectivenessSlice ■■■ batchReleaseSlice ■■■ complaintSlice ■■■ supplierSlice ■■■ aiSlice ■■■ notificationSlice

The AI slice should store transient review state such as processing status, extraction, evidence, recommendation and human edits. Final approved business values belong in their domain slice, not permanently in a generic AI bucket.

16. Environment Variables

```
# Frontend VITE_API_BASE_URL= VITE_SUPABASE_URL= VITE_SUPABASE_ANON_KEY= # Backend DATABASE_URL=
SUPABASE_URL= SUPABASE_SERVICE_ROLE_KEY= GROQ_API_KEY= GROQ_MODEL= EMBEDDING_MODEL=
STORAGE_BUCKET= JWT_AUDIENCE= JWT_ISSUER= # Runtime APP_ENV=development LOG_LEVEL=INFO
AI_TIMEOUT_SECONDS=30 MAX_UPLOAD_MB=20
```

Secrets belong only on the backend. The browser receives public client configuration, not server-side database credentials, model keys, or service-role keys.

17. Security Architecture

18. Audit and Observability Flow

USER ACTION ↓ request_id generated ↓ application log ↓ business operation ■■■ DB change ■■■ audit_event ↓ if AI: ■■■ ai_interaction ■■■ ai_evidence_links ↓ response ↓ request correlation remains searchable

19. Testing Strategy

19.1 Critical test cases

User from Company A cannot fetch Company B's deviation.

User without closure permission cannot close a deviation.

A deviation cannot move directly from Reported to Closed if required investigation/CAPA/effectiveness gates are unmet.

Repeated AI processing can be identified using request/idempotency keys.

INTERNET ██████████ ▼▼ Frontend Hosting API Hosting React static build FastAPI ██████████
PostgreSQL/Supabase █████ Supabase Auth █████ Supabase Storage █████ pgvector █████ Groq ███████ HTTPS
██████████

For a 24-hour competition window, keep deployment simple. The priority is a reliable end-to-end demo, not building a multinational cloud empire with twelve Kubernetes clusters and one exhausted developer.

Bound retrieval top-k and prompt size.

Every transition should have a service-level guard. The frontend may hide unavailable actions, but the backend must enforce the rule because browsers are not trustworthy little angels.

REQUEST POST /api/v1/ai/deviation/process request_id=req-9f2 AUTH user=qa.user company=company-01 site=site-01 STORAGE file=temperature_log.pdf storage_ref=tenant/01/deviations/temp-uuid.pdf AI parse → text extracted ocr_used=false extract → 84°C / 76–80°C validate → valid rag → SOP-014 v3.2, batch record rules → out_of_range=true llm → major / potential impact validate → valid REVIEW user decision=edited severity=major note="Confirmed by QA" PERSIST deviation=DEV-2026-018 status=open AUDIT deviation.created ai.reviewed

26. Final 5–10 Minute Video Flow

The supplied challenge brief specifically asks for a QMS understanding video and the Deviation → Investigation → Root Cause → CAPA → Closure connection. Keep the demo centered on that path rather than attempting to demonstrate every conceivable QMS feature.

27. Final Technical Acceptance Checklist

- Frontend loads and authenticates.
- Tenant isolation works.
- Batch can be opened.
- In-process check can be viewed/created.
- Deviation can be created.
- AI extraction can process the demo document.
- RAG evidence appears with source metadata.
- Human can accept/edit/reject AI recommendation.
- Investigation can be started from deviation.
- Root cause can be drafted and confirmed.
- CAPA can be created and actions assigned.
- Effectiveness can be recorded.
- Deviation can only close through valid workflow state.
- Batch Release screen shows quality context.
- Complaint/supplier context is linked.
- Audit events appear.
- AI interaction metadata is persisted.
- Failure paths do not create false success.
- Demo seed data tells one coherent story.
- Video can be completed in 5–10 minutes.

28. Final Architecture Rule

Build the smallest real system that proves the complete QMS chain. The evaluator does not need a sprawling enterprise implementation. They need to see that the API manufacturing processes connect, that a deviation drives investigation and root cause, that CAPA follows from the cause, that effectiveness and closure are controlled, and that AIVOA's AI assists the workflow with evidence and human review.

END OF FILE 5

Layer	Recommended technology	Purpose
Frontend	React + Vite + TypeScript	QMS screens and workflow UI
State	Redux Toolkit	Shared batch/deviation/investigation/CAPA state
UI	Tailwind CSS + component library	Consistent enterprise UI
Backend	FastAPI + Python	REST APIs and orchestration
Validation	Pydantic	Request/response and AI output validation
ORM	SQLAlchemy	PostgreSQL persistence
Migrations	Alembic	Versioned database schema
Database	PostgreSQL / Supabase	Transactional QMS data
Vector search	pgvector	RAG evidence retrieval
Auth	Supabase Auth	Identity and session management
Storage	Supabase Storage	Uploaded records/documents
AI orchestration	LangGraph	Multi-step stateful AI workflows
LLM	Groq via existing integration	Extraction/reasoning/summarization
Document parsing	PyMuPDF / existing parser	Digital PDF text extraction
OCR	Existing OCR fallback	Scanned document extraction
Deployment	Existing project deployment + managed DB	Fast competition-ready delivery

Check	Purpose
Authentication	Identify the signed-in user
Company membership	Prevent cross-company data access

Check	Purpose
Site access	Limit site-specific records
Role/permission	Control actions such as approval/closure
Entity ownership	Verify record belongs to current tenant
Audit actor	Record who initiated the action

HTTP	Use
400	Invalid request/business input
401	Missing or invalid authentication
403	Authenticated but not authorized
404	Entity does not exist in accessible scope
409	State/version conflict
422	Schema validation failure
429	Rate/usage limit
500	Unexpected server error
503	AI/dependency temporarily unavailable

Step	Demo record	System behavior
1	Supplier: ChemCorp	Supplier context exists
2	Raw material: Acetic Anhydride / RM-2026-001	Material linked to supplier
3	Batch: API-2026-041	Manufacturing batch created
4	Step 3: Reaction	Process context shown
5	IPC: Temperature = 84°C	Expected range 76–80°C
6	DEV-2026-018	Deviation created
7	INV-2026-012	Investigation opened
8	RCA	Inadequate preventive maintenance candidate/confirmed after review
9	CAPA-2026-009	Corrective + preventive actions
10	Effectiveness	Next 5 batches monitored
11	Closure	Human-approved final closure
12	Batch Release	Quality workflow visible before disposition
13	Complaint context	Downstream quality connection shown

Area	Implementation
Identity	Supabase Auth/JWT
Tenant isolation	company_id derived from authenticated membership
Authorization	role + site + record checks
Database	PostgreSQL constraints/RLS where used
Storage	tenant-scoped paths + signed access where appropriate
Secrets	server environment only
AI inputs	untrusted document content treated as data
Audit	immutable-style append-only event records
API abuse	rate limits / bounded AI requests
Uploads	size/type validation and safe parsing

Event	Example
deviation.created	DEV-2026-018 created
deviation.ai_processed	AI extraction/recommendation completed
ai.reviewed	QA accepted/edited recommendation
investigation.created	INV-2026-012 opened
root_cause.confirmed	RCA finalized
capa.created	CAPA-2026-009 created

Event	Example
effectiveness.recorded	Monitoring result recorded
deviation.closed	Deviation closed
batch.release.reviewed	Batch release review completed

Level	Examples
Unit	Pydantic schemas, state transitions, risk rules
AI unit	Prompt output validation, malformed JSON handling
RAG	Metadata filters, relevant evidence retrieval
API	Auth, CRUD, state transition endpoints
Integration	Deviation → investigation → CAPA chain
Frontend	Workflow stepper, forms, review panels
E2E	Upload → AI → review → closure
Security	Cross-tenant access attempts
Regression	Existing Round 1 deviation workflow

Environment	Purpose
Local	Developer implementation and seeded demo
Staging/demo	Evaluator-ready stable workflow
Production-like	Optional hardened deployment if available

Decision	Why
React + Redux	Matches existing Round 1 direction and supports complex workflow state
FastAPI	Fast typed Python API layer suitable for AI orchestration
PostgreSQL	Relational model fits linked QMS records and auditability
Supabase	Auth/storage/Postgres convenience while preserving PostgreSQL core
pgvector	Keeps RAG retrieval close to QMS data
LangGraph	Makes multi-step AI workflows explicit and stateful
Groq	Preserves existing Round 1 LLM integration direction
Pydantic	Strong contract between AI output and business logic
SQLAlchemy + Alembic	Controlled persistence and schema migration

Phase	Work	Expected result
1	Lock DB migrations and seed data	Connected API batch story exists
2	Build Batch + IPC context	Manufacturing context visible
3	Upgrade Deviation UI/API	Round 1 remains functional
4	Add Investigation + RCA	Core investigation flow works
5	Add CAPA + effectiveness	Core CAPA lifecycle works
6	Connect AI graph	Extraction/RAG/recommendation works
7	Add human review	Accept/Edit/Reject is explicit
8	Add Batch Release + lightweight complaint/supplier	Full QMS relationship visible
9	Seed one polished demo story	Evaluator can follow one batch
10	Run E2E + record video	Submission ready